

The Complete Guide to Replication

Copies of data, and the honest cost of each one: CAP and PACELC without the folklore, replication topologies, quorums, consensus, clocks, and what multi-region actually promises.

Consistency, per operation

Linearizable to eventual as a ladder — and the sentence that stops the CAP argument.

Topologies and their failures

Single-leader, multi-leader, leaderless; lag, failover, split brain and fencing.

Quorums and repair

$W + R > N$, sloppy quorums, hinted handoff, read repair, Merkle anti-entropy.

Consensus and clocks

Raft in one page, $2f+1$, what a majority costs, and why timestamps are not order.

What's inside

Read it front to back once. Part 1 gives you the language, Part 3 the arithmetic and Part 6 the multi-region answer interviewers now expect from senior candidates.

1 Consistency, said precisely

CAP and PACELC properly, the consistency ladder from linearizable to eventual, and per-operation guarantees.

2 Replication topologies

Single-leader, multi-leader and leaderless; what gets shipped; sync vs async; lag, failover, split brain and fencing.

3 Quorums and tunable consistency

$W + R > N$, sloppy quorums and hinted handoff, read repair, anti-entropy, and conflict resolution when writes collide.

4 Consensus

Why you need it, Raft in one page, $2f+1$, what a majority costs, and the systems that give it to you.

5 Time, order and clocks

Clock skew, wall vs monotonic, Lamport and hybrid logical clocks, TrueTime, and why a timestamp is not an ordering.

6 Multi-region

Active-passive vs active-active, the 80 ms tax, data residency, RPO and RTO, and failover you have actually rehearsed.

7 The interview itself

How to answer a consistency question, 14 asked-in-real-interviews Q&A, and the red flags to avoid.

★ One-page cheat sheet

The night-before page: the ladder, the quorum arithmetic, the numbers, eight facts.

Part of the HLD concept series. Sharding (splitting data), databases (the engine) and messaging (moving events) are separate volumes; this one is only about copies and agreement.

How to use this guide

Understand

Each mechanism gets the plain-English version, then the failure it exists to survive.

Say out loud

The grey “interview line” boxes are the sentences that end a CAP argument instead of starting one.

Defend

Every guarantee has a latency price. Interviewers score whether you named it.

THE WORDS PEOPLE MISUSE

Replication makes copies for availability and read capacity. **Sharding** splits data for write capacity.

Consensus gets several machines to agree on one value. **Consistency** in CAP means linearizability, and consistency in ACID means “your invariants hold” — they are different words wearing the same clothes, and saying so is worth a point on its own.

THE ONE THING TO INTERNALISE

Every consistency guarantee is bought with **coordination**, and coordination is paid for in **latency and availability**. So the design question is never “strong or eventual?” — it is “**which operations are worth a round trip, and which are not?**”

Consistency, said precisely

PART 1

Most candidates can recite CAP and almost none can use it. The fix is to stop treating consistency as a system-wide setting and start treating it as a property of each operation.

1.1 CAP, said correctly

CAP is about behaviour **during a network partition**: when nodes cannot talk, you either refuse the request (choose consistency) or answer possibly-stale (choose availability). It is not a general classification of databases, and “we are a CP system” as a blanket statement is the tell that someone learned it from a slide.

PACELC is the better frame: *if there is a Partition, choose A or C; Else, choose Latency or Consistency*. That second half is the everyday trade-off — partitions are rare, but the choice between waiting for a quorum and answering from the nearest replica is made on every single request.

1.2 The ladder

stronger — more coordination, more latency

weaker — cheaper, faster, more surprises

Linearizable

every read sees the latest committed write, globally

money, inventory, unique-name claims

Sequential / causal

related events keep their order; unrelated ones may not

chat messages, comment threads

Read-your-writes

you always see your own updates

any user-facing write: profile, post

Monotonic reads

time never runs backwards for one session

feeds, timelines, dashboards

Eventual

replicas converge, eventually, with no ordering promise

counters, likes, view counts, analytics

Pick per operation, not per system.

Figure 1.1 — *The consistency ladder. Every step down removes a round trip and adds a class of surprise you must design around.*

1.3 The session guarantees that users actually notice

Users do not experience linearizability; they experience their own edit disappearing. Three cheap guarantees fix almost every user-visible anomaly without global coordination:

Read-your-writes

After I write, I see it. Pin my reads to the primary for a few seconds, or require a replica at least as fresh as my write's log position.

Monotonic reads

Time never runs backwards for me. Stick a session to one replica so a second read cannot land on a staler one.

Consistent prefix

I never see an answer before its question. Replicate related writes in order, which usually means same partition or same stream.

Causal consistency

The general version of the above: writes that depend on each other are seen in order. Implemented with logical clocks or dependency tracking.

INTERVIEW LINE

"Different operations in this system deserve different guarantees. The payment and the inventory decrement are linearizable, because getting those wrong costs money. The order history is read-your-writes, so a user always sees what they just did. The view counter is eventual and I would not spend a round trip on it."

1.4 What "strong consistency" costs, in numbers

same-datacenter quorum ~ 1–2 ms · cross-AZ ~ 1–3 ms · cross-region ~ 80–100 ms

A linearizable write inside one region is affordable. The same write coordinated across continents turns a 5 ms API into a 200 ms API — which is why global systems keep writes regional and replicate asynchronously.

Replication topologies

PART 2

Three shapes, and each one is a different answer to “who is allowed to accept a write?”. Everything else — lag, conflicts, failover — follows from that.

2.1 The three shapes

Topology	How writes work	What it buys · what it costs
Single-leader	All writes to one node, which ships the change to followers	No write conflicts, simple reasoning · one write bottleneck, and a failover window when the leader dies
Multi-leader	Several nodes accept writes and replicate to each other	Local writes in every region, survives a region · conflicts are guaranteed and you must resolve them
Leaderless (Dynamo-style)	The client (or a coordinator) writes to W replicas and reads from R	No failover at all, tunable per request · read repair, anti-entropy and version conflicts become your problem

2.2 What actually gets shipped

Statement replication

Ship the SQL. Cheap and dangerous: NOW(), RAND() and triggers diverge. Largely abandoned.

Write-ahead-log / physical

Ship the byte-level changes. Exact and fast, but the replica must run the same engine version — this is Postgres streaming replication.

Logical / row-based

Ship “row X changed to Y”. Version-tolerant, works across engines, and it is what CDC pipelines consume.

Trigger-based

Application-level, flexible, slow. Only when nothing else can see the change.

2.3 Sync, async, and the honest middle

Synchronous means the write is not acknowledged until a replica has it: zero data loss, and your write latency now includes the slowest replica — plus writes stop entirely if that replica is down. **Asynchronous** acknowledges immediately: fast, and a failover can lose the tail. **Semi-synchronous** is what most production systems actually run: one replica synchronous, the rest async, so you can lose a node without losing data and without waiting for everybody.

2.4 Replication lag and the three anomalies it causes

“Where did my edit go?”

The write went to the leader, the read to a follower. Fix: read-your-writes — pin to the leader briefly, or require a replica fresh enough by log position.

“Time went backwards.”

Two reads hit two followers with different lag. Fix: monotonic reads — stick a session to one replica.

The third is the one people forget: **an answer arriving before its question**, when two related writes replicate through different paths. Keep causally related writes on one partition or one stream, and you have consistent prefix reads for free.

2.5 Failover, split brain and fencing

Failover is a sequence, and every step can hurt: detect the leader is gone (too eager and you flap, too slow and you are down), choose a new leader (ideally the most up-to-date replica), redirect clients, and decide what to do about writes the old leader accepted but never replicated.

- **Split brain** is two nodes both believing they are the leader — the failure that produces silent, permanent divergence.
- **Fencing** is the fix: a monotonically increasing epoch or term number that storage refuses to accept writes below, so a resurrected old leader is rejected rather than trusted.
- **STONITH / lease-based leadership** — the leader holds a time-bounded lease and must stop serving when it cannot renew it.
- **The data question:** under async replication, promoting a replica means discarding the old leader’s unreplicated tail. Someone has to decide that deliberately; say so.

INTERVIEW LINE

“Automatic failover with async replication means I am choosing availability over those last few writes. I would make that explicit: fence with an epoch number so the old primary cannot come back and write, quote an RPO of a few seconds, and have a reconciliation path for anything the application cannot lose.”

Quorums and tunable consistency

PART 3

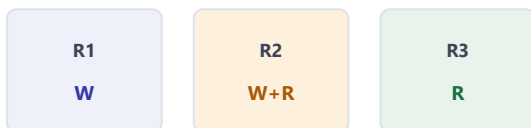
The leaderless model turns consistency into arithmetic you can do on a whiteboard, which is exactly why interviewers like it.

3.1 $W + R > N$

With N replicas, a write acknowledged by W of them and a read answered by R of them are guaranteed to overlap when $W + R > N$ — so the read sees the latest write. Everything else in Dynamo-style tuning is a consequence of choosing those three numbers.

$N = 3$ replicas: the write set and the read set must overlap

$W = 2, R = 2 \rightarrow 2 + 2 > 3$



the sets share R2, so the read sees the write

$W = 1, R = 1 \rightarrow 1 + 1 < 3$



no overlap: the read can miss the write

The three settings you should be able to justify

$W=N, R=1$ — fast reads, slow and fragile writes: one replica down blocks writing.

$W=1, R=1$ — fastest and weakest: eventual, and the classic “I read stale data” bug.

$W=2, R=2$ ($N=3$) — the default. Survives one node down, and a read overlaps the last write.

Figure 3.1 — Quorum overlap with $N = 3$. The arithmetic is the guarantee; the settings are the dial.

3.2 What breaks quorums, and the repairs

Sloppy quorum When the “right” replicas are unreachable, write to any W healthy nodes instead. Availability up, and the data is now in the wrong place.	Hinted handoff Those stand-in nodes hold a hint and forward the data when the real owner returns. This is what makes sloppy quorums safe rather than lossy.
Read repair A read that sees disagreement writes the newest value back to the stale replica. Repair paid for by traffic — so cold data never gets fixed this way.	Anti-entropy A background job compares Merkle trees of key ranges and syncs the differences. This is what repairs the data nobody reads.

3.3 When writes collide

Resolution	How it decides	When it is acceptable
Last write wins (LWW)	Highest timestamp survives; the other is discarded	Only when losing a write is genuinely fine — and remember clock skew decides the winner
Vector clocks / version vectors	Detects that two versions are siblings rather than ordered	When the application can merge, or ask the user (the shopping-cart answer)
CRDTs	A merge function that is commutative, associative and idempotent	Counters, sets, collaborative text — anything whose merge rule can be fixed in advance
Application merge	Your code reconciles siblings on read	The general escape hatch: correct, and it is code you must actually write

The senior framing: “**last-write-wins is not conflict resolution, it is conflict deletion — so I would only use it where I can name the write I am willing to lose.**”

Consensus

PART 4

Consensus is how a group of machines agrees on one value despite failures. You will rarely implement it and you are frequently asked to explain it, because everything that has a single leader depends on it.

4.1 What it is for

- **Electing a leader** — and making sure there is only ever one per term.
- **Agreeing on a log** — an ordered sequence every node replays to reach the same state (this is state-machine replication).
- **Holding configuration and locks** — topology, shard maps, leases; the job etcd, ZooKeeper and Consul exist to do.
- **Committing transactions across partitions** — Spanner, CockroachDB and TiDB run a consensus group per shard.

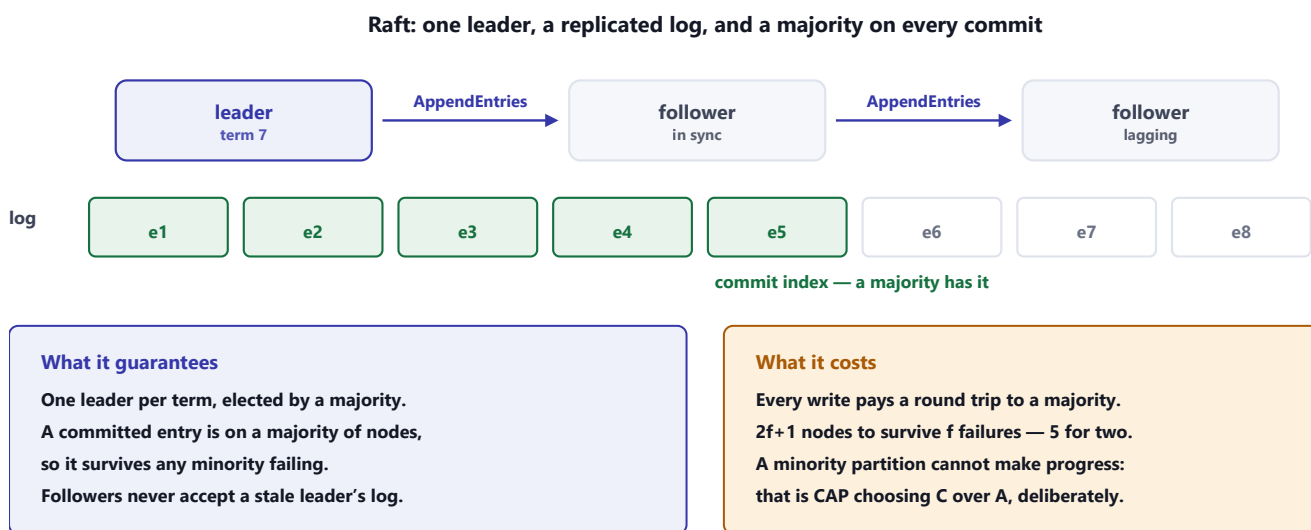
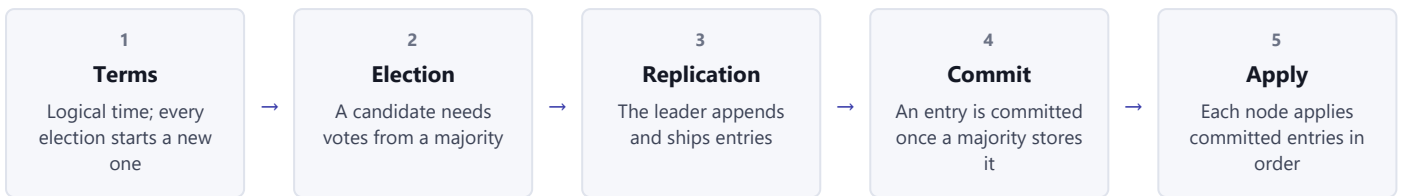


Figure 4.1 — Raft in one picture: a leader per term, entries committed once a majority has them, and a minority that cannot make progress on purpose.

4.2 Raft in one page



Two properties make it safe: a candidate can only win with a majority (so two leaders in one term are impossible), and a voter refuses a candidate whose log is behind its own (so a committed entry can never be overwritten). Membership changes go through the same log — joint consensus — which is why adding a node is a committed entry, not a config file edit.

4.3 The arithmetic and the cost

survive f failures with $2f + 1$ nodes · 3 nodes tolerate 1 · 5 tolerate 2

Every commit costs a round trip to a majority, so latency is the median replica, not the fastest. Bigger groups survive more and commit slower — which is why production clusters are three or five, never nine.

Paxos / Multi-Paxos

The original. Same guarantees, famously harder to describe — say “Raft is Paxos made teachable” and move on.

ZAB

ZooKeeper’s protocol: atomic broadcast with a leader, tuned for configuration workloads.

Kafka KRaft

Kafka’s own Raft implementation for metadata, which is why it no longer needs ZooKeeper.

The honest limitation

Consensus gives you agreement, not throughput. It is for metadata and leadership — do not put every user write through one Raft group.

Time, order and clocks

PART 5

Half of the hard bugs in replicated systems are someone trusting a timestamp. This part is the vocabulary to avoid that, and it is a reliable senior differentiator.

5.1 Why wall-clock time lies

- **Skew:** NTP-synced machines are typically within a few milliseconds, but hundreds of milliseconds happen — and a virtual machine can pause for seconds.
- **Steps:** a clock can jump backwards when it is corrected, so “later” timestamps can be produced before earlier ones. Use a monotonic clock for durations, never the wall clock.
- **Leap seconds and VM pauses:** the classic sources of a lease you think you hold and no longer do.
- **The consequence:** ordering by timestamp is a heuristic. Under LWW conflict resolution, clock skew silently decides which write survives.

5.2 Logical clocks

Clock	What it tells you	Cost
Lamport clock	A total order consistent with causality	Cannot distinguish concurrent from ordered
Vector clock	Whether two events are causally ordered or concurrent	Size grows with the number of writers
Hybrid logical clock (HLC)	Causality plus a timestamp close to real time — the modern default in CockroachDB and YugabyteDB	Still bounded by clock skew, but degrades safely
TrueTime (Spanner)	An interval with a bounded error, backed by GPS and atomic clocks	Commit waits out the uncertainty window — correctness bought with hardware

THE SENTENCE THAT SCORES

“I would not order events by wall-clock timestamp across machines. For causality I would use a logical clock, and if I need last-write-wins I would say out loud that clock skew is choosing the winner — which is only acceptable where losing that write is acceptable.”

Multi-region

PART 6 · THE 2026 EXPECTATION

Interviewers now expect a senior candidate to reason about regions unprompted: where writes land, what replicates, what a failover loses, and where the data is legally allowed to live.

6.1 The three shapes

Active-passive	Active-active, partitioned	Active-active, everywhere
• writes in one region, async copy • RPO: seconds of loss • RTO: minutes, needs a promotion • simple, and the default answer	• each region owns a key range • no write conflicts by construction • cross-region reads are stale • the pattern most large systems use	• any region takes any write • conflicts are guaranteed: LWW or CRDT • or consensus per write, at 80–100 ms • only worth it if the product demands it

A cross-region round trip is 80–100 ms. That single number decides the shape: anything on the write path that needs another region is a user-visible cost, so keep writes local and replicate asynchronously.

Figure 6.1 — Three multi-region shapes. The cross-region round trip is the number that decides between them.

6.2 Say it as a table, per data class

Data	Where it is written	How it replicates
Payments, ledger	One home region per account	Synchronous within the region, async across — never a cross-region write on the hot path
User profile, settings	Home region of the user	Async global copies; read locally, accept seconds of staleness
Session, cache	Local region only	Not replicated at all. Rebuild on failover; that is what a cache is for
Catalogue, config	One writer, global read	Async fan-out to every region; it changes rarely and is read constantly

6.3 RPO, RTO, and rehearsing the failover

RPO — recovery point

How much data you accept losing. Async cross-region replication makes this “the unreplicated tail”, usually seconds.

RTO — recovery time

How long until you serve again. DNS or global load-balancer switching plus a promotion: minutes if rehearsed, hours if not.

The drill

A failover plan that has never been executed is a document, not a capability. Saying you would run game days is a cheap, credible point.

The asymmetry

Failing over is the easy half. Failing *back* — reconciling writes taken while the primary was away — is the part teams forget.

6.4 Data residency, in one paragraph

Some data may not leave a jurisdiction. That turns into a sharding-and-replication rule: the user’s home region is part of their identity, personal fields never replicate out, and derived or anonymised data is what travels. It is worth one sentence in any design with European or Indian users, and it is often the real reason a system is geo-partitioned rather than globally replicated.

6.5 What “eventually consistent” must mean in practice

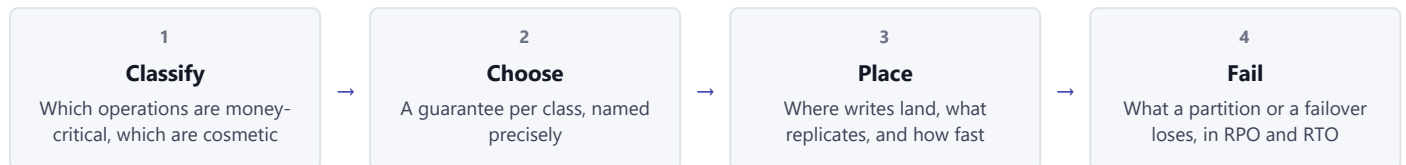
- **A bound:** “replicas converge within a second under normal conditions” — and an alert when lag exceeds it.
- **A user-visible story:** which screens can be stale, and what the product does about it (optimistic UI, “syncing...”, or hiding the operation entirely).
- **A repair path:** read repair plus anti-entropy, so divergence is corrected rather than accumulated.
- **A measurement:** replication lag as a first-class SLI, per region and per partition, because it is the number your failover decision depends on.

The interview itself

PART 7

Consistency questions are traps for the unprepared because the honest answer is always “it depends” — and the score comes from what it depends on.

7.1 The four-step answer



7.2 The five sentences that earn points

- “CAP only bites during a partition; the everyday trade is PACELC’s second half — latency versus consistency.”
- “Different operations deserve different guarantees, so here is my per-operation table.”
- “W plus R greater than N gives me overlap; with N=3 I would run W=2, R=2 and survive one node.”
- “Last-write-wins deletes a write and lets clock skew choose which one — I would only accept that for this field.”
- “Cross-region consensus costs 80 to 100 milliseconds per write, so writes stay regional and replicate asynchronously.”

INTERVIEW LINE

“I would keep the strong guarantees inside one region and one partition, because that is where coordination is cheap: a quorum inside a datacentre is a couple of milliseconds. Everything global gets asynchronous replication with an explicit staleness budget, and the operations that cannot tolerate that are exactly the ones I would keep pinned to the user’s home region.”

7.3 Fourteen questions with model answers

Q1 Explain CAP, and then tell me what is wrong with how people use it.

During a network partition you can either refuse requests to stay consistent or serve possibly-stale data to stay available. What is wrong is treating it as a permanent label: partitions are rare, so most of the time you are choosing between latency and consistency instead — that is PACELC, and it is the trade you make on every request.

Q2 What does $W + R > N$ actually guarantee?

That the set of replicas you wrote to and the set you read from must overlap in at least one node, so the read sees the latest acknowledged write. With $N=3$ and $W=R=2$ you get that overlap and you survive one replica being down. It does not give you linearizability in general — concurrent writes, sloppy quorums and read repair all complicate it — which is a good nuance to volunteer.

Q3 A user updates their profile and sees the old value. Diagnose and fix.

Replication lag: the write went to the leader and the read to a follower that has not applied it. The fix is read-your-writes — route that user's reads to the leader for a few seconds, or carry the write's log position and require a replica at least that fresh. Add sticky routing for monotonic reads so the value cannot flip back and forth.

Q4 Sync or async replication — which do you choose?

Semi-synchronous in almost every case: one replica acknowledges synchronously so a single node failure loses nothing, and the rest follow asynchronously so the write does not wait for the slowest. Fully synchronous means a slow replica becomes your write latency and a dead replica blocks writes; fully async means a failover discards the tail. I would state the RPO either way.

Q5 How does leader election work, and what is split brain?

A consensus protocol — Raft or similar — has candidates request votes for a new term and requires a majority, which makes two leaders in one term impossible. Split brain is when that guarantee is missing or ignored and two nodes both act as leader, so writes diverge permanently. The defence is fencing: a monotonic epoch or term number that storage checks, so a returning old leader is rejected instead of trusted.

Q6 Why not use consensus for everything?

Because every committed write pays a round trip to a majority, and a minority partition cannot make progress at all. That is the right trade for metadata, leadership and configuration, and the wrong one for a high-volume data path. Systems that do use it for data — Spanner, CockroachDB — run one consensus group per shard, and they still keep the group inside a region wherever possible.

Q7 Two regions accepted conflicting writes. What now?

First, note that active-active everywhere makes this inevitable, so it must be a design decision rather than a surprise. Then pick the resolution honestly: last-write-wins if I can name the write I am willing to lose; version vectors if the application can merge or ask the user; a CRDT if the merge rule can be fixed in advance. Or avoid it entirely by partitioning ownership so each key has exactly one writing region.

Q8 Is “eventually consistent” an acceptable answer?

Only with three things attached: a bound (“converges within a second, alerting above five”), a user-visible story for what can be stale, and a repair path — read repair plus anti-entropy — so divergence is corrected rather than accumulated. Without those it is a way of saying “I have not thought about it”.

Q9 How do you detect and repair divergence between replicas?

Two mechanisms working together. Read repair fixes what traffic touches: a read that sees disagreement writes the newest value back. Anti-entropy fixes what nobody reads: a background job compares Merkle trees of key ranges and syncs only the ranges whose hashes differ, which is $\log(n)$ messages rather than a full comparison.

Q10 Design the consistency model for a payment system.

Linearizable where money moves: the ledger write and the balance check go through one partition with a real transaction, and idempotency keys make retries safe. Read-your-writes for the user's own transaction list. Eventual for aggregates and dashboards. Cross-region, the account has a home region and writes never leave it — I would rather fail a write over than take an 80 ms coordination penalty or risk a conflicting balance.

Q11 Why can't you order events by timestamp across machines?

Because clocks disagree — a few milliseconds normally, far more after a VM pause or a correction — and they can even step backwards, so a "later" event can carry an earlier timestamp. For causality use logical clocks: Lamport for a consistent total order, vector clocks to detect concurrency, hybrid logical clocks if you also want the value to look like a real time. Spanner solves it with TrueTime and pays for it in hardware.

Q12 How many nodes in a consensus cluster, and why?

Three or five. You need $2f+1$ to survive f failures, so three tolerates one and five tolerates two. Beyond that every commit gets slower because a majority is larger, while the marginal availability gain is tiny. Even numbers buy nothing — four nodes still only tolerate one failure and make quorums bigger.

Q13 Walk me through a region failover.

Detect with a health signal that is not the failing region's own; shift traffic at DNS or the global load balancer; promote the replica database with a fence so the old primary cannot write; accept and record the RPO — the writes that never replicated; and expect a cold cache, so ramp traffic rather than shifting all of it at once. Then the hard part, which is failing back and reconciling what happened while the primary was away.

Q14 What is the difference between ACID consistency and CAP consistency?

ACID's C means your declared invariants hold after a transaction — it is about constraints. CAP's C means linearizability: every read sees the latest write, as if there were one copy. A single Postgres node gives you ACID consistency and, being one copy, trivially the CAP kind too; a replicated eventually-consistent store can be fully ACID per node and not linearizable across the cluster.

7.4 Red flags interviewers listen for

Saying this	Says this about you
"We're a CP system" as a general property	Learned CAP from a slide; it only applies during a partition
"Strong consistency everywhere"	Has not priced a cross-region round trip
"Eventually consistent" with no bound or repair path	A way of saying "I have not thought about it"
Last-write-wins presented as conflict resolution	It deletes a write, and clock skew picks the loser
Ordering distributed events by wall-clock timestamp	Has not met clock skew
Automatic failover with no fencing	Split brain and permanent divergence
"Add a replica" to fix a write bottleneck	Replicas do not take writes
A consensus group on the user write path	Correct and slow; consensus is for metadata and leadership
Active-active without naming conflict resolution	Conflicts are guaranteed, not possible
No RPO/RTO numbers in a failover answer	Has never been on call for one

7.5 Real systems worth name-dropping

Amazon Dynamo (2007)

Leaderless replication, W/R/N tuning, sloppy quorums, hinted handoff, vector clocks and Merkle repair. Every quorum answer traces back to this paper.

Google Spanner (2012)

External consistency at global scale with TrueTime and Paxos groups. The honest counterexample to "you cannot have both" — paid for in hardware and commit waits.

Raft (Ongaro & Ousterhout, 2014)

Consensus written to be understandable, and now the implementation in etcd, Consul, TiKV, CockroachDB and Kafka's KRaft.

Jepsen

Kyle Kingsbury's test reports showing real databases violating the guarantees they advertised. Citing it signals you distrust marketing claims.

One-page cheat sheet

The night-before page. The arithmetic and the latencies are what make the answer concrete.

The numbers

quorum in one DC ~ 1–2 ms · cross-AZ ~ 1–3 ms · cross-region ~ 80–100 ms · $2f + 1$ to survive f

$W + R > N$ gives read/write overlap. Three or five nodes in a consensus group. Async replication makes RPO “the unreplicated tail”; synchronous makes it zero and adds the slowest replica to every write.

The decisions

Question	Default answer
Consistency level?	Per operation. Linearizable for money, read-your-writes for user edits, eventual for counters
Topology?	Single-leader inside a region; partitioned ownership across regions
Sync or async?	Semi-sync: one synchronous replica, the rest async
N, W, R?	$N=3$, $W=2$, $R=2$ — overlap plus one-node tolerance
Conflicts?	Avoid by ownership; then CRDT or version vectors; LWW only where a lost write is acceptable
Consensus for what?	Leadership, membership, configuration, locks — not the data path
Cluster size?	3 or 5. Never even, never nine
Ordering?	Logical clocks, not wall clocks
Failover?	Fenced by epoch, rehearsed, with RPO and RTO written down
Multi-region?	Writes local, replication async, staleness budget stated

Replication in eight facts

CAP is about partitions only

The everyday trade is latency versus consistency — PACELC's second half.

Coordination is the price

Every guarantee costs a round trip. Keep the expensive ones inside one region.

Replicas add reads, not writes

A write bottleneck is a sharding problem, never a replica problem.

Lag is a feature you must design for

Read-your-writes and monotonic reads fix the anomalies users actually notice.

$W + R > N$ is the whole quorum idea

$N=3$, $W=2$, $R=2$ by default. Anything less is a deliberate weakening.

Failover without fencing is divergence

An epoch number storage enforces is what stops a returning old leader.

$2f + 1$ survives f

Three or five nodes. Consensus is for metadata, not for every write.

Timestamps are not order

Clock skew decides LWW winners. Use logical clocks for causality.

The failure modes

Name	Mechanism	Primary fix
Stale read	Read hit a lagging replica	Read-your-writes; freshness requirement by log position
Time travel	Two reads, two replicas, different lag	Sticky session to one replica (monotonic reads)
Split brain	Two nodes both believe they lead	Fencing epoch enforced at the storage layer
Lost tail	Async failover discards unreplicated writes	Semi-sync, or accept and state the RPO
Silent divergence	Conflicting writes resolved by LWW	Version vectors or CRDTs; ownership per key
Repair never runs	Read repair only fixes what is read	Merkle-tree anti-entropy on a schedule

IF YOU SAY NOTHING ELSE, SAY THIS

“Consistency is bought with coordination, and coordination is paid for in latency and availability — so I choose it per operation rather than per system. Money is linearizable inside one region and one partition; a user’s own edits are read-your-writes; counters are eventual with a stated bound and a repair path. Across regions writes stay local and replicate asynchronously, because a cross-region round trip is eighty milliseconds I refuse to put on the hot path. And failover is fenced, rehearsed, and quoted with an RPO and an RTO.”

Sources & further reading — Martin Kleppmann, *Designing Data-Intensive Applications*, chapters 5, 7 and 9; DeCandia et al., *Dynamo* (SOSP 2007); Ongaro & Ousterhout, *In Search of an Understandable Consensus Algorithm* (Raft, 2014); Corbett et al., *Spanner* (OSDI 2012); Daniel Abadi on PACELC; Gilbert & Lynch’s CAP proof and Eric Brewer’s *CAP Twelve Years Later*; Kyle Kingsbury’s Jepsen analyses; Kulkarni et al. on hybrid logical clocks; Shapiro et al. on CRDTs; the etcd, ZooKeeper and Cassandra documentation on quorums, leases and repair.